Green University of Bangladesh
Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering

Semester: (Spring, Year:2026), B.Sc. in CSE (Day)

Lab Report NO #03

Course Title: Database Lab

Course Code: CSE 210 Section: 242-D3

Lab Experiment Name: Design and Implementation of University Database Using SQL JOIN Operations and Aggregate Functions.

Student Details

Name		ID
1.	Ashik Arman Ziku	221002245

Submission Date : 16/04/2026

Course Teacher's Name : Md. Sabbir Hosen Mamun

Lab Report Status

Marks: …… **Signature:.....**

Comments:..... **Date:.....**

1. TITLE OF THE LAB REPORT EXPERIMENT

Design and Implementation of University Database Using SQL JOIN Operations and Aggregate Functions.

2. OBJECTIVES / AIM

- To create multiple related tables in a relational database
- To establish relationships using Foreign Keys
- To perform JOIN operations (INNER JOIN, LEFT JOIN)
- To retrieve combined data from multiple tables
- To use aggregate functions with GROUP BY and HAVING
- To analyze student performance using SQL queries.

3. THEORY/PROCEDURE/Algorithm

A university database system was designed with four tables:

- DEPARTMENT
- STUDENT
- COURSE
- ENROLLMENT

Relationships

- One DEPARTMENT has many STUDENTS
- One DEPARTMENT offers many COURSES
- STUDENT enrolls in COURSE through ENROLLMENT

Algorithm

Part A: Database and Table Creation

1. Create database university_join_lab
2. Create DEPARTMENT, STUDENT, COURSE, and ENROLLMENT tables

Part B: Data Insertion

1. Insert at least 4 records into DEPARTMENT
2. Insert at least 8 records into STUDENT
3. Insert at least 6 records into COURSE
4. Insert at least 12 records into ENROLLMENT

Part C: Query Execution

1. Perform INNER JOIN queries

2. Perform LEFT JOIN queries
3. Apply GROUP BY and HAVING
4. Use aggregate functions
5. Filter data using conditions

4. IMPLEMENTATION

```
CREATE DATABASE university_join_lab;
USE university_join_lab;

CREATE TABLE DEPARTMENT (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(100) NOT NULL UNIQUE,
   Building VARCHAR(100) NOT NULL
);

CREATE TABLE STUDENT (
   StudentID INT PRIMARY KEY,
   FullName VARCHAR(100) NOT NULL,
   DeptID INT,
   Batch VARCHAR(20) NOT NULL,
   CGPA DECIMAL(3,2),
   FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
);

CREATE TABLE COURSE (
   CourseID INT PRIMARY KEY,
   CourseTitle VARCHAR(100) NOT NULL,
   Credit INT CHECK (Credit > 0),
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
);

CREATE TABLE ENROLLMENT (
   EnrollID INT PRIMARY KEY,
   StudentID INT,
   CourseID INT,
   Semester VARCHAR(20) NOT NULL,
   Marks INT CHECK (Marks >= 0 AND Marks <= 100),
   FOREIGN KEY (StudentID) REFERENCES STUDENT(StudentID),
   FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)
);

INSERT INTO DEPARTMENT VALUES
(1, 'CSE', 'A Building'),
(2, 'EEE', 'B Building'),
(3, 'BBA', 'C Building'),
(4, 'ENG', 'D Building');
```

```
INSERT INTO STUDENT VALUES
(1, 'Sakib Hasan', 1, '2021', 3.75),
(2, 'Arif Hossain', 2, '2020', 3.50),
(3, 'Sumaiya Akter', 1, '2022', 3.90),
(4, 'Nusrat Jahan', 3, '2021', 3.20),
(5, 'Shanto Das', 4, '2019', 3.10),
(6, 'Tanvir Ahmed', 2, '2020', 3.60),
(7, 'Salman Khan', 1, '2021', 3.80),
(8, 'Rafiul Islam', 3, '2022', 3.30);

INSERT INTO COURSE VALUES
(1, 'Database Systems', 3, 1),
(2, 'Circuit Theory', 3, 2),
(3, 'Marketing', 3, 3),
(4, 'English Literature', 3, 4),
(5, 'Data Structures', 3, 1),
(6, 'Digital Electronics', 3, 2);

INSERT INTO ENROLLMENT VALUES
(1, 1, 1, 'Spring 2024', 85),
(2, 1, 5, 'Spring 2024', 78),
(3, 2, 2, 'Spring 2024', 65),
(4, 2, 6, 'Spring 2024', 70),
(5, 3, 1, 'Spring 2024', 92),
(6, 3, 5, 'Spring 2024', 88),
(7, 4, 3, 'Spring 2024', 75),
(8, 5, 4, 'Spring 2024', 60),
(9, 6, 2, 'Spring 2024', 82),
(10, 7, 5, 'Spring 2024', 90),
(11, 8, 3, 'Spring 2024', 68),
(12, 7, 1, 'Spring 2024', 87);

-- 1. Student name with department name
SELECT s.FullName, d.DeptName
FROM STUDENT s
INNER JOIN DEPARTMENT d ON s.DeptID = d.DeptID;

-- 2. Student name, course title, semester, marks
SELECT s.FullName, c.CourseTitle, e.Semester, e.Marks
FROM ENROLLMENT e
INNER JOIN STUDENT s ON e.StudentID = s.StudentID
INNER JOIN COURSE c ON e.CourseID = c.CourseID;

-- 3. All students including those not enrolled
SELECT s.FullName, e.CourseID
FROM STUDENT s
LEFT JOIN ENROLLMENT e ON s.StudentID = e.StudentID;

-- 4. All courses including no enrollment
SELECT c.CourseTitle, e.StudentID
```

```
FROM COURSE c
LEFT JOIN ENROLLMENT e ON c.CourseID = e.CourseID;

-- 5. Department-wise total students
SELECT d.DeptName, COUNT(s.StudentID) AS TotalStudents
FROM DEPARTMENT d
LEFT JOIN STUDENT s ON d.DeptID = s.DeptID
GROUP BY d.DeptName;

-- 6. Course-wise total enrolled students
SELECT c.CourseTitle, COUNT(e.StudentID) AS TotalEnrolled
FROM COURSE c
LEFT JOIN ENROLLMENT e ON c.CourseID = e.CourseID
GROUP BY c.CourseTitle;

-- 7. Avg, Max, Min marks per course
SELECT c.CourseTitle,
    AVG(e.Marks) AS AvgMarks,
    MAX(e.Marks) AS MaxMarks,
    MIN(e.Marks) AS MinMarks
FROM COURSE c
JOIN ENROLLMENT e ON c.CourseID = e.CourseID
GROUP BY c.CourseTitle;

-- 8. Each student's total courses and avg marks
SELECT s.FullName,
    COUNT(e.CourseID) AS TotalCourses,
    AVG(e.Marks) AS AvgMarks
FROM STUDENT s
LEFT JOIN ENROLLMENT e ON s.StudentID = e.StudentID
GROUP BY s.FullName;

-- 9. Top 3 students by average marks
SELECT s.FullName, AVG(e.Marks) AS AvgMarks
FROM STUDENT s
JOIN ENROLLMENT e ON s.StudentID = e.StudentID
GROUP BY s.FullName
ORDER BY AvgMarks DESC
LIMIT 3;

-- 10. Departments having more than 2 students
SELECT d.DeptName, COUNT(s.StudentID) AS TotalStudents
FROM DEPARTMENT d
JOIN STUDENT s ON d.DeptID = s.DeptID
GROUP BY d.DeptName
HAVING COUNT(s.StudentID) > 2;

-- 11. Students with marks between 60 and 85
SELECT s.FullName, e.Marks
FROM STUDENT s
JOIN ENROLLMENT e ON s.StudentID = e.StudentID
```

```
WHERE e.Marks BETWEEN 60 AND 85;

-- 12. Students name starts with 'S' or contains 'a'
SELECT FullName
FROM STUDENT
WHERE FullName LIKE 'S%' OR FullName LIKE '%a%';
```

5. TEST RESULT / OUTPUT

```
SELECT s.FullName, d.DeptName FROM STUDENT s INNER JOIN DEPARTMENT d ON s.DeptID = d.DeptID;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

FullName	DeptName
Sakib Hasan	CSE
Arif Hossain	EEE
Sumaiya Akter	CSE
Nusrat Jahan	BBA
Shanto Das	ENG
Tanvir Ahmed	EEE
Salman Khan	CSE
Rafiul Islam	BBA

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Query results operations

Figure 01 : Inner join Depertment Name from Student

```
SELECT s.FullName, c.CourseTitle, e.Semester, e.Marks FROM ENROLLMENT e INNER JOIN STUDENT s ON e.StudentID = s.StudentID INNER JOIN COURSE c ON e.CourseID = c.CourseID;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

FullName	CourseTitle	Semester	Marks
Sakib Hasan	Database Systems	Spring 2024	85
Sumaiya Akter	Database Systems	Spring 2024	92
Salman Khan	Database Systems	Spring 2024	87
Arif Hossain	Circuit Theory	Spring 2024	65
Tanvir Ahmed	Circuit Theory	Spring 2024	82
Nusrat Jahan	Marketing	Spring 2024	75
Rafiul Islam	Marketing	Spring 2024	68
Shanto Das	English Literature	Spring 2024	60
Sakib Hasan	Data Structures	Spring 2024	78
Sumaiya Akter	Data Structures	Spring 2024	88
Salman Khan	Data Structures	Spring 2024	90
Arif Hossain	Digital Electronics	Spring 2024	70

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Figure 02 : Fullname, CourseTitle, Semester and Mark

```
SELECT s.FullName, e.CourseID FROM STUDENT s LEFT JOIN ENROLLMENT e ON s.StudentID = e.StudentID;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

FullName	CourseID
Sakib Hasan	1
Sakib Hasan	5
Arif Hossain	2
Arif Hossain	6
Sumaiya Akter	1
Sumaiya Akter	5
Nusrat Jahan	3
Shanto Das	4
Tanvir Ahmed	2
Salman Khan	5
Rafiul Islam	3
Salman Khan	1

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Figure 03 : FullName with CourseId

```
SELECT c.CourseTitle, e.StudentID FROM COURSE c LEFT JOIN ENROLLMENT e ON c.CourseID = e.CourseID;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

CourseTitle	StudentID
Database Systems	1
Data Structures	1
Circuit Theory	2
Digital Electronics	2
Database Systems	3
Data Structures	3
Marketing	4
English Literature	5
Circuit Theory	6
Data Structures	7
Marketing	8
Database Systems	7

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Figure 04 : CourseTitle according to StudentId

```
SELECT d.DeptName, COUNT(s.StudentID) AS TotalStudents FROM DEPARTMENT d LEFT JOIN STUDENT s ON d.DeptID = s.DeptID GROUP BY d.DeptName;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

DeptName	TotalStudents
BBA	2
CSE	3
EEE	2
ENG	1

Show all | Number of rows: 25 Filter rows: Search this table

Figure 05 : DeptName according to StudentId

```
SELECT c.CourseTitle, COUNT(e.StudentID) AS TotalEnrolled FROM COURSE c LEFT JOIN ENROLLMENT e ON c.CourseID = e.CourseID GROUP BY c.CourseTitle;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

CourseTitle	TotalEnrolled
Circuit Theory	2
Data Structures	3
Database Systems	3
Digital Electronics	1
English Literature	1
Marketing	2

Show all | Number of rows: 25 Filter rows: Search this table

Figure 06 : CourseTitle according to Total Enrolled

```
SELECT c.CourseTitle, AVG(e.Marks) AS AvgMarks, MAX(e.Marks) AS MaxMarks, MIN(e.Marks) AS MinMarks FROM COURSE c JOIN ENROLLMENT e ON c.CourseID = e.CourseID GROUP BY
c.CourseTitle;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

CourseTitle	AvgMarks	MaxMarks	MinMarks
Circuit Theory	73.5000	82	65
Data Structures	85.3333	90	78
Database Systems	88.0000	92	85
Digital Electronics	70.0000	70	70
English Literature	60.0000	60	60
Marketing	71.5000	75	68

Show all | Number of rows: 25 Filter rows: Search this table

Figure 07 : CourseTitle according to MaxMarks and MinMarks

```
SELECT s.FullName, COUNT(e.CourseID) AS TotalCourses, AVG(e.Marks) AS AvgMarks FROM STUDENT s LEFT JOIN ENROLLMENT e ON s.StudentID = e.StudentID GROUP BY s.FullName;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

FullName	TotalCourses	AvgMarks
Arif Hossain	2	67.5000
Nusrat Jahan	1	75.0000
Rafiul Islam	1	68.0000
Sakib Hasan	2	81.5000
Salman Khan	2	88.5000
Shanto Das	1	60.0000
Sumaiya Akter	2	90.0000
Tanvir Ahmed	1	82.0000

Show all | Number of rows: 25 Filter rows: Search this table

Figure 08 : CourseTitle according to AvgMarks

```
SELECT s.FullName, AVG(e.Marks) AS AvgMarks FROM STUDENT s JOIN ENROLLMENT e ON s.StudentID = e.StudentID GROUP BY s.FullName ORDER BY AvgMarks DESC LIMIT 3;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Extra options

FullName	AvgMarks 1
Sumaiya Akter	90.0000
Salman Khan	88.5000
Tanvir Ahmed	82.0000

Query results operations

Print | Copy to clipboard | Export | Display chart | Create view

Figure 09 : CourseTitle according to AvgMarks as descending in limit 3.

```
SELECT s.FullName, e.Marks FROM STUDENT s JOIN ENROLLMENT e ON s.StudentID = e.StudentID WHERE e.Marks BETWEEN 60 AND 85;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

FullName	Marks
Sakib Hasan	85
Sakib Hasan	78
Arif Hossain	65
Arif Hossain	70
Nusrat Jahan	75
Shanto Das	60
Tanvir Ahmed	82
Rafiul Islam	68

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Figure 10 : Student Marks between 60 to 85.

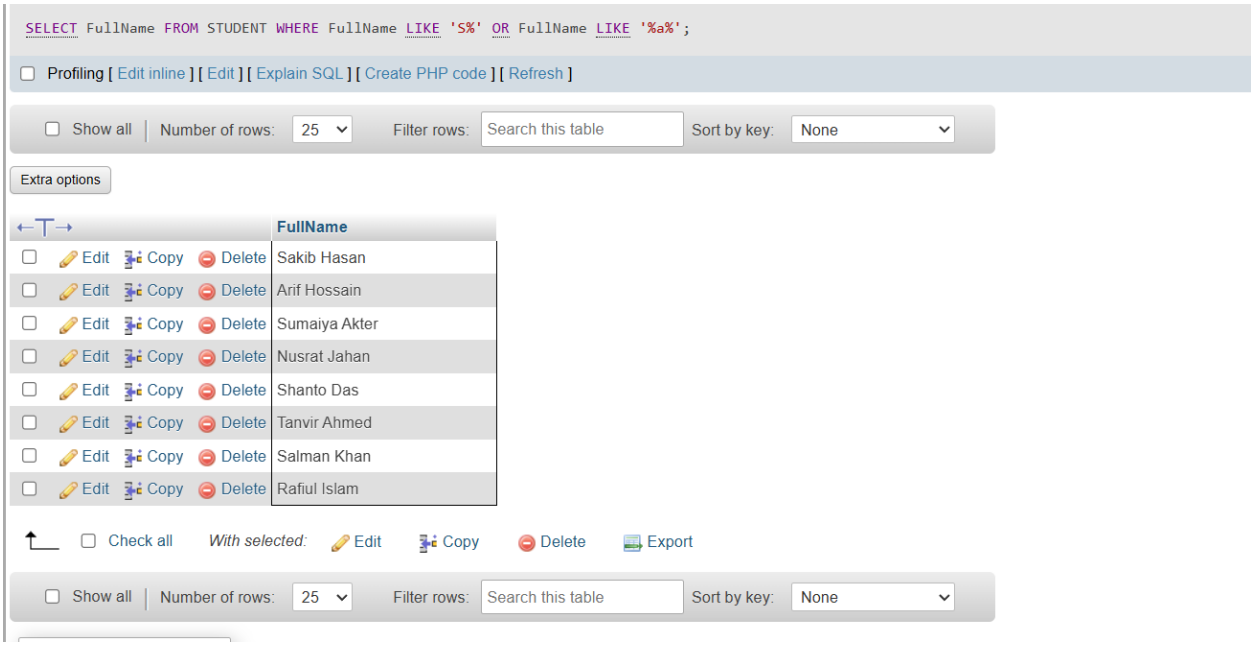

Figure 11 : FullName from Student.

6. ANALYSIS AND DISCUSSION

- INNER JOIN combines matching records from multiple tables
- LEFT JOIN includes unmatched records
- GROUP BY helps summarize data
- HAVING filters grouped data
- Aggregate functions provide statistical insights
- JOIN operations are essential for relational databases.

7. SUMMARY

In this experiment, a university database named university_join_lab was created with four related tables: DEPARTMENT, STUDENT, COURSE, and ENROLLMENT. Relationships were established using foreign keys. Various JOIN operations were performed to retrieve combined data from multiple tables. Aggregate functions and grouping techniques were used to analyze student performance and course data. This experiment enhanced understanding of SQL joins, relational database design, and data analysis.